

The μ OS++ test framework

Install, build, run and add tests, step by step

Kernel: micro-os-plus-iii

Ports: aarch32, aarch64, cortexm, posix-arch

The μ OS++ test framework, step by step

- 1. What the framework is
- 2. Install, once
- 3. Build one configuration
- 4. Run tests
- 5. Add a test to a board
- 6. Add a harness suite
- 7. Run a harness suite as a cortexm board app
- 8. Add a platform or a configuration
- 9. When something goes wrong
- 10. What exists today
- 11. Rules to keep
- 12. HOWTO

The μOS++ test framework, step by step

This guide explains how to install the test framework, build it, run the tests, and add new ones. Each step says **what** you do and **how** you do it. Every command here was checked against the code.

For the common jobs as short recipes (install, build debug and release, run, add a test, a suite, a board, a configuration or a whole project), and for how the `.json` files drive it all, go straight to [§12 HOWTO](#).

1. What the framework is

The framework has four parts:

- **CTest** registers every test and runs it.
- **CMake** builds the test images.
- **xpm**, the xPack tool, gives each build its own pinned compiler and tools, and has named commands for every step. These are called **actions**.
- **VS Code**, with the xPack plugin, shows those actions as buttons.

All of it lives in the unified `micro-os-plus-iii/` repository (branch `smp`) and its `tests/` directory.

A test tells the framework its result in one of two ways:

- it prints a line `RESULT: PASS`, `RESULT: FAIL` or `RESULT: SKIP`;
- or it exits with a code, where 0 means pass.

Three files, all edited by hand, decide what exists:

File	What it decides
<code>tests/package.json</code>	The configurations : which platform, compiler and build type. Also the actions , the commands you run.
<code>tests/cmake/tests-main.cmake</code>	Which port a platform tests, chosen by the platform name's prefix.
<code>tests/platforms/<platform>/CMakeLists.txt</code>	Which tests that platform registers in CTest.

Words used below

Word	Meaning
platform	One board or emulator setup, e.g. <code>aarch32-rpi-zero-2w</code> , <code>cortexm-pico2</code> , <code>native</code> .
configuration (<code>C</code>)	A platform plus a compiler plus a build type, e.g. <code>cortexm-pico2-cmake-gcc-debug</code> . Each one has its own folder, <code>build/C/</code> .
case	One CTest test, named <code><platform>-<test>-<variant></code> . A harness suite is registered as <code><platform>-<suite>-test</code> (e.g. <code>qemu-cortex-m0-rtos-apis-test</code>), with no variant suffix.
variant	Where the test runs. <code>qemu</code> runs in the emulator, <code>host</code> runs on the PC, and <code>hwd</code> runs on the real board.
action	A named command in <code>package.json</code> , run as <code>xpm run <action> --config C</code> .
board test	A test that belongs to one board. It lives in the port: <code><port>/test/<board>/<test>/</code> .
harness suite	A portable test with no <code>main()</code> . It lives in <code>tests/sources/<suite>/</code> and runs on many platforms.

2. Install, once

2.1 The workspace folder

What: one folder that holds the six repositories side by side. The default is `~/Work`, but any folder works (this machine also uses `~/TMP`).

How:

```
mkdir -p ~/Work/micro-os-plus && cd ~/Work/micro-os-plus
# The unified kernel & test framework:
git clone "$HOME/Downloads/GIT/micro-os-plus-iii.git" micro-os-plus-iii
# The architecture ports and devices:
for r in aarch32 aarch64 cortexm devices posix-arch; do
    git clone "$HOME/Downloads/GIT/micro-os-plus-iii-$r.git" \
        "micro-os-plus-iii-$r"
done
```

The framework and ports look for siblings under both standard folder names (`micro-os-plus-iii-<port>`) and names with a `.git` suffix (`micro-os-plus-iii-<port>.git`). No symlinks are needed.

2.2 The tools

What: Node packages, plus the xPack packages CMake, Ninja, QEMU, the build helper and the test libraries.

How:

```
cd ~/Work/micro-os-plus/micro-os-plus-iii
npm install
xpm install
```

All the commands in this guide can be run from the repository root or its `tests/` folder.

3. Build one configuration

What: four steps, always in this order:

Step	Command	What it does
1	<code>xpm run install --config C</code>	Puts the pinned compiler into <code>build/C/xpacks/</code> .
2	<code>xpm run prepare --config C</code>	Runs CMake with that compiler and finds the tests.
3	<code>xpm run build --config C</code>	Compiles every test image.
4	<code>xpm run test --config C</code>	Runs every emulated and host test. It skips hardware tests.

Example:

```
C=aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run install --config $C
xpm run prepare --config $C
xpm run build --config $C
xpm run test --config $C
```

Why the order matters:

- `prepare` needs the compiler that `install` puts there. If you skip `install`, CMake picks the PC's own compiler (`found /usr/bin/cc`).
- Only `prepare` tells CMake which compiler to use. Skip it and `build` may use the wrong one.
- `test` does **not** build. Run it on a tree that was never built and every test fails with `no *-host executables`.

Debug and release are separate. Each has its own `build/C/` folder, so a `-release` configuration needs its own `install`, too.

Never delete `build/C/`, because it holds the pinned compiler. If CMake got confused, delete only `build/C/CMakeCache.txt` and run `prepare` again.

4. Run tests

I want to...	Command
run all emulated and host tests of one configuration	<code>xpm run test --config C</code> (build first)
run one test; it builds whatever it needs first	<code>xpm run test-<test>-<variant> --config C</code>
see which tests exist	<code>PATH="\$PWD/build/C/xpacks/.bin:\$PATH" ctest --test-dir build/C -N</code>
run one case by hand	same <code>PATH</code> , then <code>ctest --test-dir build/C -V -R <platform>-<test>-<variant></code>
run debug and release in one go	<code>xpm run test-<platform>-cmake</code> , for every platform with emulated or host tests: <code>native</code> , <code>2xcortex-m33</code> , <code>pico2-1cpu</code> , <code>cortexm-pico2</code> , <code>cortexm-pico2-rp2350b-psram</code> , the four Raspberry Pi boards, and the four <code>qemu-cortex-m*</code> (also via <code>test-cortex-cmake</code>).

Examples:

```
xpm run test-smp_test0-qemu      --config aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run test-rtos-apis-qemu     --config cortexm-pico2-cmake-gcc-debug
xpm run test-cmsis-os-validator-host --config native-cmake-gcc-debug
```

From VS Code: open `tests/` and pick a configuration in the xPack view. The plugin shows **actions**, not CTest cases. A test appears there only if its `test-*` action exists in `package.json`.

On real hardware (`hwd`):

- The board must be connected.
- Run one test per power cycle.
- The test goes through OpenOCD only, with no GDB. The result comes over the UART or semihosting.
- The generic `test` action skips `hwd` cases on purpose. Run a hardware case by its own action, e.g. `xpm run test-smp_test0-hwd --config aarch32-rpi-zero-2w-cmake-gcc-debug`.

Emulated suites run one at a time, on a quiet machine. When several run in parallel they get slow and can time out for no real reason.

Where the logs are: next to the images, in `.qemu-logs/`, `.host-logs/` or `.hw-logs/`.

5. Add a test to a board

What: a new folder with one test in the port, then an action so VS Code shows it. The platform registers the test by **scanning the port's test folders** when `prepare` runs. You do **not** edit any CMake file in the framework.

How:

1. **Write it.** Create `<port>/test/<board>/<name>/main.cpp`, starting from a sibling test. It must print `RESULT: PASS` or `RESULT: FAIL`. On hardware it must also **stop** after that line: call `hw_result::ok()` / `hw_result::fail()` (cortexm) or `semihosting::report_result()`. Otherwise the run never ends.
2. **Declare what it needs**, only if it differs from the default. Add it to the lists in `<port>/test/<board>/tests.cmake`:
`BOARD_TEST_NEED_DEVICES`, `BOARD_TEST_SELF_CONTAINED`,
`BOARD_TEST_NO_KERNEL`, and on cortexm `BOARD_TEST_HWD_ONLY` (no emulator image).
3. **Give it time**, only if it runs longer than 300 s in QEMU. Add it to `timeout_for` in `micro-os-plus-iii-smp.git/test_smpl/run-qemu.sh`.
4. **Commit it.** If you wrote it in another copy of the port, push it there, then `git pull` in the workspace copy.
5. **Re-scan:** `xpm run prepare --config C`. Without this step the test does not exist in CTest.
6. **Add the action.** Run `python3 ~/.claude/skills/micro-os-xpack-tests/check-actions.py . C --emit`, which prints the missing `test-<name>-<variant>` lines. Paste them into the `actions` of **every** configuration that builds the test. Nothing generates these lines for you.
7. **Check and run:** `sh python3 ~/.claude/skills/micro-os-xpack-tests/check-actions.py . PATH="$PWD/build/C/xpacks/.bin:$PATH" cmake --build build/C -- -k 0 xpm run test-<name>-qemu --config C` The full build matters. Every per-test action builds the **whole** tree, so one broken test breaks every action of that configuration.

After adding the test, run the emulated set of the other boards on the same port again. A new test must not change their results.

6. Add a harness suite

What: a portable test that runs on many platforms. It has no `main()`. Its entry point is `int os_main(int argc, char* argv[])`, and it returns 0 for pass.

How:

1. **Create** `tests/sources/<name>/`, copying `mutex-stress`:
2. `CMakeLists.txt` defines the library `test::<name>`;
3. `src/*.cpp` holds the test;
4. `include/cmsis-plus/os-app-config.h` holds its RTOS settings.

5. **Make it known to a platform.** Add `"${CMAKE_SOURCE_DIR}/sources/<name>"` to `tests/platforms/<platform>/cmake/dependencies-folders.cmake`.
6. **Register it.** Add `add_harness_suite(<name> <name>-test)` to that platform's `CMakeLists.txt`. The helper differs by family (see §12.6): the Pi platforms use the port's own builder, `pico2-1cpu`, `2xcortex-m33` and the four `qemu-cortex-m*` use `add_suite_executable`, and only `nucleo-*/raspberrypi-pico` use `add_test_executable`.
7. **Re-scan, add the action and run it**, as in steps 5–7 of section 5.

The suite links `micro-os-plus::platform-support`, which supplies `main()` and the start-up hooks. A board test must **not** link it, because it has its own `main()`.

Suites today: `mutex-stress`, `rtos-apis`, `cmsis-os-validator`, `fp-switch`, plus `blinky` and `instrumentation` (`nucleo-f411re` only). The validator raises NVIC IRQ 0: on Cortex-M that is the NVIC, on `native` the validator xpack's signal shims, and on the four Raspberry Pi platforms the BCM2837's local Mailbox 1 (see `TESTS-CATALOG.md` §4).

7. Run a harness suite as a cortexm board app

What: run a suite on a cortexm board, on the real board and, where the board has a QEMU image, in QEMU too. `cortexm-pico2`, `cortexm-nucleof411`, `cortexm-weactf411` and `cortexm-weactf412` already do this for `rtos-apis`, `mutex-stress` and `cmsis-os-validator`. To do it on another board, copy these five pieces. **All five are needed.**

1. **The verdict wrapper, once per board**, `test/<board>/harness-suite.hpp`, and **a folder per suite**, `test/<board>/<suite>/harness-suite.cpp`, which only includes `../harness-suite.hpp`. The folder name becomes the app name. The header defines `__wrap_os_main()`: it runs the suite, prints `RESULT: PASS` or `RESULT: FAIL`, and returns the code. (The Pi ports keep the wrapper in each suite's `harness-suite.cpp` instead.)
2. `board_test_libs()` in `tests.cmake` returns the suite library, its extras, and an interface target (on pico2, `pico2-harness-suite`). That target carries:
3. the defines `OS_USE_OS_APP_CONFIG_H` `UOS_HARNESS_SUITE` `_POSIX_C_SOURCE=200809L` `_GNU_SOURCE`;
4. the platform's `include/` folder;
5. the semihosting, newlib and POSIX-io libraries;
6. the link option `-Wl,--wrap=os_main`.
7. **The board's `os-app-config.h`** hands over to the suite's own settings when `UOS_HARNESS_SUITE` is set. Each of the board's own values is wrapped in `#ifndef`.
8. **The port's builder** already builds the emulated image with one CPU, because the generic QEMU core is single-core. You don't need to change it.
9. **One action per case** in `package.json`, as in step 6 of section 5.

8. Add a platform or a configuration

A platform:

1. Copy the closest folder in `tests/platforms/`.
2. The name must start with a prefix that `tests/cmake/tests-main.cmake` knows: `aarch32`, `aarch64`, `native`, `cortexm`, `qemu-cortex`, `pico2` or `2xcortex`. Otherwise add a new branch there.
3. Set the board in `cmake/definitions.cmake`: `set (BOARD "<id>" CACHE STRING "" FORCE)`.

A port platform's `CMakeLists.txt` does five things:

1. loads the port's board facts;
2. reuses the port's own test builder;
3. scans `<port>/test/<board>/*`;
4. registers one case per test and variant;
5. adds the harness suites.

A configuration:

1. In `tests/package.json`, copy the nearest debug and release pair.
2. Change `platformName`, `toolchainFileName` and `shortConfigurationName`. The short name must be unique.
3. Pick the tools it inherits: `qemu-arm-dependencies` for an emulator, `openocd-dependencies` for a board.
4. Set its `test` action to `ctest -V -LE hwd`.
5. Add its `test-*` actions. If it has emulated tests, also add a top-level `test-<platform>-cmake`.

The release entry only inherits the debug one and changes `buildType`.

Then follow section 3 with `install → prepare → build → test`.

9. When something goes wrong

What you see	Why, and the fix
<code>found /usr/bin/cc</code> , or the compiler must be 15.2	CMake ran before <code>install</code> , or <code>build</code> ran before <code>prepare</code> . Run <code>install</code> , then <code>prepare</code> .
Missing <code>.../build/C/xpacks/.../CMakeLists.txt</code> on a release configuration	The release folder never had its own <code>install</code> .
<code>no *-host executables</code> , or every case fails	<code>test</code> does not build. Run <code>build</code> first, or use a per-test action.
The test runs in CTest but is missing in VS Code	Its <code>test-*</code> action is missing. Run <code>check-actions.py</code> .
Every <code>test-*</code> action of one configuration fails while building	One test in that tree does not compile. Run <code>cmake --build build/C -- -k 0</code> to see them all.
A new port test is missing from <code>ctest -N</code>	You didn't run <code>prepare</code> again, the folder has no <code>.cpp</code> file, or the workspace copy wasn't pulled.
<code>alias micro-os-plus::iii already exists</code>	The kernel was added twice. The port already adds it.
<code>undefined reference to os_startup_initialize_hardware...</code>	A suite is missing <code>platform-support</code> .
A QEMU case times out with no error	Tests ran in parallel, or the test needs a <code>timeout_for</code> entry. Run it again alone first.
A hardware test never ends	It prints <code>RESULT</code> but never stops. Add <code>hw_result</code> or <code>report_result</code> .

10. What exists today

There are **22 platforms** and **70 configurations** (35 debug + 35 release; `native` has one configuration per compiler). The numbers below are the debug case counts from `ctest -N`. What every test does, its scheduling mode and the probe each board uses are in `TESTS-CATALOG.md`.

Platform	Tests	Emulated / host	Hardware
<code>aarch32-rpi-zero-2w</code>	12 port tests + <code>mutex-stress</code> , <code>rtos-apis</code> , <code>cmsis-os-validator</code>	15 qemu	15
<code>aarch64-rpi-zero-2w</code>	the same	15 qemu	15
<code>aarch32-rpi3b</code>	the same	15 qemu	14 (no <code>usb_test</code> : the Pi 3 B's USB is behind a hub)
<code>aarch64-rpi3b</code>	the same	15 qemu	14 (likewise)
<code>aarch32-luckfox-lyra</code>	19 port tests + <code>mutex-stress</code>	–	20
<code>native</code> (POSIX, on the PC)	15 port tests + <code>cmsis-os-validator</code>	16 host	–
<code>cortexm-pico2</code>	12 port tests + the 3 suites and <code>fp-switch</code> as board apps	6 qemu	16
<code>cortexm-pico2-rp2350b-psram</code>	14 port tests	2 qemu	14
<code>cortexm-pico2-pizero</code>	14 port tests	–	14
<code>cortexm-nucleof411</code>	<code>mos-test1</code> + the 3 suites	–	4
<code>cortexm-weactf411</code>	<code>mos-test1</code> , <code>spi-pipeline</code> + the 3 suites	–	5
<code>cortexm-weactf412</code>	<code>mos-test1</code> , <code>uart-test1</code> + the 3 suites	–	5
<code>pico2-1cpu</code> (1 × Cortex-M33)	the 3 suites + <code>fp-switch</code>	4 qemu	–
<code>2xcortex-m33</code> (2 × Cortex-M33, SMP)	the 3 suites + <code>fp-switch</code>	4 qemu	–
<code>qemu-cortex-m0 / m3 / m4f / m7f</code>	the 3 suites	3 qemu each	–
<code>nucleo-f411re</code>	3 suites + <code>blinky</code> (<code>instrumentation</code> built, not registered)	–	4
<code>nucleo-f767zi</code> , <code>nucleo-h743zi</code> , <code>raspberrypi-pico</code>	the 3 suites	–	3 each

Verified passing (what has been run and seen to pass, not everything that is registered):

- the 3 suites on `qemu-cortex-m0/m3/m4f/m7f`, `pico2-1cpu` and `2xcortex-m33`, with `cmsis-os-validator` at 60/60;
- `cortexm-pico2` 6/6 and `cortexm-pico2-rp2350b-psram` 2/2 emulated;

- `native`: all 16 host cases, in `gcc` and `sys`, debug and release;
- on 2026-09-26, every local configuration's emulated/host set, run one configuration at a time; on the boards, `cortexm-pico2` 16/16 and `cortexm-pico2-rp2350b-psram` 14/14;
- `cmsis-os-validator` 60/60 on the four Raspberry Pi platforms, emulated and on the boards, debug and release;
- on the Raspberry Pi Zero 2 W platforms, `mutex-stress`, `rtos-apis` and `smp_test0`, plus `smp_test1` and `smp_test3` on AArch32.

The other cases are registered but were not all run one by one. Every hardware case builds, and runs only when its board is connected.

11. Rules to keep

- **The code is the truth.** If a document disagrees with the files, the files win. The older documents in `docs/tests/` describe an earlier layout.
 - **Commit new files with** `git add -A`. `git commit -a` leaves new folders out, and a fresh clone would then miss them.
 - **Never commit** `build/`, `tests/xpacks/` or `node_modules/`. They are regenerated by `install`, `prepare` and `build`.
 - A new board or test **must not change** results on boards that are already tested. Run their emulated sets again.
 - Always pass the **full** case name to `-R`. It matches substrings, so a short name can run more than you meant.
 - The whole history of how this framework was built is in `git log` of this file.
-

12. HOWTO

Short recipes for the common jobs. Each one gives the commands in order and points to the section above that explains them. All commands run from `micro-os-plus-iii-smp.git/tests/`, and `C` is a configuration name (`<platform>-cmake-<toolchain>-<debug|release>`).

12.1 Install the dependencies

Once per workspace: clone the six repositories side by side (§2.1), then the framework's own tools:

```
cd ~/Work/micro-os-plus-iii-smp.git/tests
npm install
xpm install
```

Once per configuration folder: its pinned compiler. Debug and release are two folders, so both need it:

```
xpm run install --config native-cmake-gcc-debug
xpm run install --config native-cmake-gcc-release
```

For the hardware (`-hwd`) configurations, `install` also brings OpenOCD; the board's probe must be plugged in only when a test runs.

12.2 Build a configuration, debug and release

```
for C in cortexm-pico2-cmake-gcc-debug cortexm-pico2-cmake-gcc-release; do
  xpm run install --config $C      # first time only
  xpm run prepare --config $C      # after adding or removing a test folder
  xpm run build --config $C
done
```

The images land in `build/C/platform-bin/...`. To see every compile error at once instead of the first one:

```
PATH="$PWD/build/C/xpacks/.bin:$PATH" cmake --build build/C -- -k 0
```

12.3 Build and run one test, debug and release

The per-test action prepares, builds the configuration and runs just that case. A release configuration inherits every action of its debug twin:

```
xpm run test-flatfs-test-host --config native-cmake-gcc-debug
xpm run test-flatfs-test-host --config native-cmake-gcc-release
```

The action name is `test-<test>-<variant>`: `-qemu`, `-host` or `-hwd`. A harness suite's action is `test-<suite>-test` (e.g. `test-rtos-apis-test`). `xpm run` with no action lists the actions of **every** configuration, each expanded to its real command; to see one configuration's:

```
xpm run 2>&1 | grep -A1 '^- native-cmake-gcc-release/'
```

Without xpm, on a configuration that was built:

```
PATH="$PWD/build/C/xpacks/.bin:$PATH" ctest --test-dir build/C -V \
  -R '^<platform>-<test>-<variant>$'
```

Anchor the name with `^...$`: `-R` matches substrings.

12.4 Run tests

Goal	Command
every emulated or host test of a configuration	<code>xpm run build --config C</code> , then <code>xpm run test --config C</code>
the same, debug and release, for a whole platform	<code>xpm run test-<platform>-cmake</code> (where it exists, §4)
one test	<code>xpm run test-<test>-<variant> --config C</code>
list the cases	<code>PATH="\$PWD/build/C/xpacks/.bin:\$PATH" ctest --test-dir build/C -N</code>
one test on the board	power-cycle the board, then <code>xpm run test-<test>-hwd --config C</code>

Rules that save time:

- **One emulated set at a time**, on an idle machine. A four-core QEMU set starved of CPU stalls with no fault and times out; run a failure again, alone, before debugging it.
- **One hardware test per power cycle**. The previous run's state (on the RP2350, its SIO spinlocks) survives a re-flash otherwise.
- **The verdict** is the `RESULT: PASS|FAIL|SKIP` line; a hardware test must also stop after it (semihosting exit), or the runner waits for its timeout.
- **Interactive tests** wait for you: `smp-test-nested-clock*` for `y` on the UART, the USB tests for the host tool or a key press (see `TESTS-CATALOG.md`). Keep your terminal on the board's UART.
- **Logs** are next to the images: `.qemu-logs/`, `.host-logs/`, `.hw-logs/<test>.log`.

12.5 Add a test to an existing board

1. Create `<port>/test/<board>/<name>/main.cpp` from a sibling test. It prints `RESULT: PASS` or `RESULT: FAIL` and, on hardware, stops (§5 step 1).
2. Only if it needs more than the default, name it in `<port>/test/<board>/tests.cmake` (`BOARD_TEST_NEED_DEVICES` for the SD card, and so on; §5 step 2). A test that talks to the SD card on `native` also needs its own image: add it to `sd_image_for` in `test_smpl/run-host.sh`.
3. Commit it in the port, and `git pull` the workspace copy if you wrote it elsewhere.
4. `xpm run prepare --config C`: the folder scan runs only here.
5. Add the actions to `package.json`, for **every debug configuration** that builds the test (release inherits them):

```
sh python3 ~/.claude/skills/micro-os-xpack-tests/check-actions.py . C --emit
```

Paste the printed lines, then check:

```
sh python3 ~/.claude/skills/micro-os-xpack-tests/check-actions.py .
```

1. Run it, debug and release (§12.3), and run the other boards' emulated sets of that port again: nothing else may change.

A worked example is the native `flatfs-test`: posix-arch `77d012d` (the test and `tests.cmake`) and kernel `c581944` (the SD image and the two actions).

12.6 Add a harness suite

A portable test with `os_main()` and no `main()`; §6 has the details.

1. Copy `tests/sources/mutex-stress/` to `tests/sources/<name>/` and rename its library to `test::<name>`.
2. For each platform that runs it: add the folder to `platforms/<platform>/cmake/dependencies-folders.cmake` and register it in `platforms/<platform>/CMakeLists.txt` with that platform's own helper (the helper differs by family: `add_harness_suite()` only on `aarch32-luckfox-lyra`; the Pi platforms build the suites through the port's own builder; `add_suite_executable()` on `pico2-1cpu`, `2xcortex-m33` and the four `qemu-cortex-m*`).
3. On a cortexm board, also give it a board app (§7).
4. `prepare`, add the actions, run (§12.5 steps 4–6).

A worked example is `fp-switch`: kernel `c41c06d` (the suite, the two platforms and the actions) and cortexm `92caefe` (the pico2 board app).

12.7 Add a board to an existing project

A board belongs to one port (project) and is described there.

1. `<port>/test/boards/<id>/board.cmake`: the board facts. The port requires `UOS_BOARD_SRC_DIR`, `UOS_BOARD_NCPU`, `UOS_BOARD_CAPS` and `UOS_BOARD_DEFINES`; a board with the `sdcard` or `usb-device` capability must also set `UOS_BOARD_DEVICES`. Copy the closest sibling; on cortexm a variant of an existing board may `include` its `board.cmake` and restate only what differs (the RP2350 boards do).
2. `<port>/test/boards/<id>/`: its start-up sources and its runner, `hw.sh` for a board, `qemu.sh` or `run.sh` for an emulator or the host.
3. `<port>/test/<id>/`: one folder per test, and `tests.cmake` for the knobs.
4. A platform for it in the framework, `tests/platforms/<port-prefix>-<id>/` (§8), with `set (BOARD "<id>" ...)` in `cmake/definitions.cmake`.
5. Its configurations (§12.8), then §12.2 and §12.4.

12.8 Add a configuration

In `tests/package.json`, under `xpack.buildConfigurations`:

1. Copy the nearest **debug** entry, e.g. `cortexm-weactf412-cmake-gcc-debug`. Set `platformName`, `toolchainFileName` and a unique `shortConfigurationName`, and pick the inherited dependencies (`qemu-arm-dependencies` for an emulator, `openocd-dependencies` for a board).
2. Set its `test` action to `ctest -V -LE hwd`, and add one `test-*` action per case (`check-actions.py ... --emit`).

3. Add the **release** entry: it only inherits the debug one and changes `buildType` and `shortConfigurationName`.
4. `python3 -c "import json; json.load(open('package.json'))"` to catch a JSON slip, then §12.1–§12.4 for both.

12.9 Add a project (a new port)

A project is one port repository, `micro-os-plus-iii-<port>.git`, cloned beside the others. Copy the closest existing port — `posix-arch` is the smallest — and keep its contract:

1. **The root** `CMakeLists.txt` finds the kernel and devices as siblings (`UOS_SMP_DIR`, `UOS_DEVICES_DIR`, accepting the `.git` suffix), adds both, selects `BOARD` among the `test/boards/*/board.cmake` it finds, loads that file and checks the required board facts (§12.7). It exports the port as an interface target with an alias `micro-os-plus::<port>`, and sets `UOS_PORT_LIB` to it (and `UOS_PORT_BARE_LIB` if some test must link no kernel).
2. `test/CMakeLists.txt` is the test builder: it includes `test/${BOARD}/tests.cmake`, loops over `test/${BOARD}/*/` and calls `uos_add_app()` (kernel `cmake/uos-app.cmake`) once per test and variant. No test name and no board name appears in it.
3. **The kernel's side:** the port's own `src/rtos/os-core.cpp` (bring-up, the kernel lock, the context switch) and its `include/cmsis-plus/rtos/port/` headers; for SMP, the contract in `docs/smp-construction.md`.
4. **The framework:** in `tests/cmake/tests-main.cmake`, a cache variable `UOS_<PORT>_DIR` pointing at `../../micro-os-plus-iii-<port>.git` and an `elseif (PLATFORM_NAME MATCHES "<prefix>")` branch that adds it; then a platform per board (§12.7 step 4) and its configurations (§12.8).
5. **The gates**, before the first commit:

```
sh cd ~/Work/micro-os-plus-iii-smp.git python3 tools/verify-no-duplicate-sources.py bash
tools/verify-no-absolute-paths.sh
```

and `tools/verify-kernel-compiles.sh` with the port's include directories (see `docs/STATUS.md`, *The three verification gates*).

12.10 Before you commit

- `check-actions.py .` reports every configuration in sync.
- The duplicate-sources gate passes.
- The emulated or host set of **every** configuration your change can reach passes, run one at a time; the boards you touched pass on hardware.
- New folders are added with `git add -A` (§11). Then push, and pull the other working copies.

12.11 The `.json` files: why, and how they work

Why JSON at all. The framework is driven by `xpm`, the xPack project manager, and `xpm` reads an npm-style `package.json` with an extra `xpack` section. That one file gives three things plain CMake does not:

- **pinned tools per build folder** — every configuration names the exact compiler, CMake, Ninja, QEMU and OpenOCD versions it uses, and `xpm install` puts them in that configuration's own folder. Nothing depends on what the PC happens to have in `/usr/bin`, and debug and release can never drift apart;
- **named commands** — the *actions* — so a long `cmake ... && ctest ...` chain is `xpm run test-flatfs-test-host --config C`, the same on every machine;
- **VS Code buttons** — the xPack extension reads the same file and shows every configuration and its actions in its side bar.

The files.

File	Role	Edit it?
<code>tests/package.json</code>	the configurations, their tools and their actions — the file this section is about	yes, by hand
<code>tests/package-lock.json</code>	npm's lock file for the Node helpers (<code>del-cli</code>), written by <code>npm install</code>	no
<code>package.json</code> (repository root)	identifies the kernel itself as the xPack <code>@micro-os-plus/micro-os-plus-iii-smp</code> : metadata only, no configurations	rarely
<code>docs/upstream-package.json</code>	upstream <code>μOS++</code> 's own <code>package.json</code> , kept for comparison	no
<code>build/C/compile_commands.json</code>	written by CMake (<code>CMAKE_EXPORT_COMPILE_COMMANDS</code>) for editors and clangd	no, generated

Inside `tests/package.json`. Beside the usual npm fields, the `xpack` object has four parts:

Key	What it holds
<code>devDependencies</code>	the tools shared by every configuration — CMake, Ninja, <code>@micro-os-plus/build-helper</code> (the CMake toolchain files), the CMSIS-RTOS validator and Chan FatFs. <code>xpm install</code> puts them in <code>tests/xpacks/</code> .
<code>properties</code>	reusable text, mostly command fragments: <code>commandCMakeReconfigure</code> , <code>commandCMakePrepareWithToolchain</code> , <code>commandCMakeBuild</code> , <code>buildFolderRelativePath</code> , ...
<code>actions</code>	the workspace-wide commands, not tied to one configuration: <code>test-native-cmake</code> , <code>test-cortex-cmake</code> , <code>test-aarch32-rpi-zero-2w-cmake</code> , <code>deep-clean</code> , ...
<code>buildConfigurations</code>	99 entries: the 70 real configurations, plus 29 marked <code>"hidden": true</code> that exist only to be inherited

How one configuration is assembled. A configuration lists what it `inherits`, then adds its own `properties` and `actions`:

```
"native-cmake-gcc-debug": {
  "inherit": [ "native-actions", "cmake-actions",
    "native-dependencies", "gcc-latest-dependencies" ],
  "properties": { "buildType": "Debug", "platformName": "native",
    "toolchainFileName": "gcc.cmake" },
  "actions": { "test": "cd {{ properties.buildFolderRelativePath }} && ctest -V -LE hwd",
    "test-flatfs-test-host": "{{ properties.commandCMakePrepareWithToolchain }} && ... -R
native-flatfs-test-host" }
}
```

The hidden entries are the building blocks:

Kind	Examples	Brings
<code>*-actions</code>	<code>cmake-actions</code> , <code>native-actions</code> , <code>cortexm-actions</code> , <code>aarch32-actions</code>	<code>install</code> , <code>prepare</code> , <code>build</code> , <code>test</code> , <code>clean</code>
<code>*-dependencies</code>	<code>gcc-latest-dependencies</code> , <code>arm-none-eabi-gcc-dependencies</code> , <code>aarch32-dependencies</code> , <code>qemu-arm-dependencies</code> , <code>openocd-dependencies</code>	the pinned compiler, QEMU or OpenOCD for that folder
<code>*-properties</code>	<code>short-win-paths-properties</code>	short build-folder names on Windows

A **release** entry inherits its debug twin whole, actions included, and overrides only `buildType` and `shortConfigurationName` — which is why the release entries list no actions of their own, and why `xpm run test-<test>-<variant> --config <...>-release` works.

How an action becomes a command. Strings are [Liquid](#) templates: `{{ ... }}` substitutes a value (`configuration.name`, `properties.*`, `os.platform`), `{% if ... %}` chooses by platform. `xpm run` expands them, then runs the result. An action that is a string is one shell command; an action that is an array runs its commands in order and stops at the first that fails. For example, `test-flatfs-test-host` on `native-cmake-sys-release` becomes:

```
cmake -S . -B build/native-cmake-sys-release -G Ninja \
-D CMAKE_BUILD_TYPE=Release -D PLATFORM_NAME=native --log-level=VERBOSE \
&& cmake --build build/native-cmake-sys-release \
&& cd build/native-cmake-sys-release && ctest -V -R native-flatfs-test-host
```

`xpm run` with no action prints every action of every configuration, expanded like this (filter it with `grep -A1 '^- C/'`) — the quickest way to see what a button will really do.

Where the tools go, and how the commands find them. `xpm run install --config C` installs that configuration's `devDependencies` into `build/C/xpacks/` (usually as links into the shared store `~/.local/xPacks/`) and their programs as links in `build/C/xpacks/.bin/`. When an action runs, xpm puts `build/C/xpacks/.bin` and `tests/xpacks/.bin` first on `PATH`, so `cmake`, `ninja`, `ctest` and the compiler are the pinned ones. That is also why every command in this guide run **without** xpm starts with `PATH="$PWD/build/C/xpacks/.bin:$PATH"`.

From JSON to CMake. The JSON never lists sources or tests. It passes three values to CMake — `PLATFORM_NAME`, `CMAKE_BUILD_TYPE` and, through the build-helper, `CMAKE_TOOLCHAIN_FILE` — and `tests/cmake/tests-main.cmake` takes over from there (§1). The ports come from the workspace (`../../micro-os-plus-iii-<port>.git`), not from `xpacks/`: a few hidden entries (`native-dependencies`, the `link-deps` actions) still name upstream's own xPacks, which the build does not use for the port.

Rules when editing it.

- Keep it valid JSON: no comments, no trailing commas. Check after every edit with `python3 -c "import json; json.load(open('package.json'))"`.
- Every `shortConfigurationName` must be unique.
- Add a `test-*` action for every new CTest case, on the debug entry only, and run `check-actions.py` (§12.5); a case without an action is invisible in VS Code.
- A change to a hidden entry reaches every configuration that inherits it; run the emulated sets of all of them.
- After changing a configuration's `devDependencies`, run `xpm run install --config C` again for its debug **and** release folders.